ChurnGuard — Project Explainer & Interview Guide

> ***ChurnGuard*** *— an end-to-end machine-learning service that predicts which telecom customers are about to churn, so a retention team can act before they leave. Repository:* `prathamshukla28/churnguard` *· License: MIT · Author: Pratham Shukla*

Part 1 — What Is ChurnGuard?

The one-line pitch

Point ChurnGuard at a customer base and it ranks every customer by churn risk, returns a calibrated probability with a plain-English *why*, and serves the whole thing as a production REST API — data, features, training, evaluation, serving, monitoring and model lifecycle in one repo.

The problem it solves

For a subscription business, keeping a customer is far cheaper than acquiring a new one. A small retention budget should be spent where it matters most, which means two things most "notebook" churn models skip:

- **Honest probabilities.** A ranking is not enough; the team needs a number that

means what it says, so "0.7" really is a ~70% chance.

- **A business decision, not 0.5.** Missing a real churner costs more than a

wasted retention offer, so the decision threshold should minimise expected cost, not default to one half.

ChurnGuard is a **scoring service, not an actuator**: it emits a calibrated probability, an explanation and a cost-based decision; a human or downstream tool takes the action.

Who it's for

Retention and growth teams at a subscription/telecom business, and the ML/MLOps engineers who have to *operate* the model — monitor it for drift, retrain it, roll it back, and keep it observable in production.

Part 2 — What It Does (the commands / API)

```
make all                      # data + train + evaluate + lint + types + tests
make serve                    # FastAPI on http://localhost:8000/docs
python -m churnguard.predict customers.csv --out scored.csv --explain
python -m churnguard.monitor recent_customers.csv    # PSI drift report
python scripts/retrain.py     # train challenger, auto-promote if better
```

The HTTP surface (FastAPI):

- `GET /` — interactive demo page.
- `GET /health` — liveness/readiness (model present? uptime).

- `GET /metrics` — the stored evaluation metrics JSON.
- `GET /metrics/prometheus` — Prometheus text exposition.
- `GET /stats` — live throughput and p50/p95/p99 latency.
- `POST /predict` — score a batch of customers, optionally with SHAP reasons.
- `POST /monitor` — drift report for a batch vs. the reference profile.

Example run

```
$ curl -s -X POST http://localhost:8000/predict \
    -H "Content-Type: application/json" \
    -d '{"explain":true,"customers":[{"tenure":1,"MonthlyCharges":95,
         "TotalCharges":95,"Contract":"Month-to-month",
         "InternetService":"Fiber optic","PaymentMethod":"Electronic check"}]}'

{"predictions":[{"churn_probability":0.97,"will_churn":true,
  "risk_band":"high","reasons":[
    {"feature":"tenure","value":1,"impact":0.9,"direction":"increases risk"},
    {"feature":"Contract","value":"Month-to-month","impact":0.6,
     "direction":"increases risk"}]}]}
```

Every prediction carries the probability, the thresholded decision, a `low`/ `medium`/`high` risk band, and (on request) the top SHAP drivers mapped back to business-readable columns.

Part 3 — Architecture

ChurnGuard is a **config-driven pipeline**: `config/config.yaml` is the single source of truth for paths, candidate models, CV folds, the cost ratio and the drift bands. Each stage is a small module with one responsibility, and a single saved artifact (`ModelBundle`) threads training time to serving time.

The lifecycle

```
          config/config.yaml  (single source of truth)
                    |
 data.py -> features.py -> train.py -------> evaluate.py
(real or   (engineering +  (CV select ->     (cost threshold,
 synthetic) ColumnTransformer) tune -> calibrate) metrics, figures, baseline)
                    |
                    v
       models/churnguard.joblib  (ModelBundle: model + threshold + SHAP bg)
                    |
          +---------+----------+
          v                    v
   explain.py (SHAP)   predict.py -> api.py  (/predict /monitor /stats ...)
```

Component	File	Responsibility
Config	`churnguard/config.py`	Typed, cached YAML loader + path resolution.
Data	`churnguard/data.py`	Synthetic generator / loader / real-data download.

Component	File	Responsibility
Features	`churnguard/features.py`	`engineer_features` + leak-free `ColumnTransformer`.
Training	`churnguard/train.py`	CV selection -> tuning -> calibration -> `ModelBundle`.
Bundle	`churnguard/bundle.py`	The one saved artifact (model + threshold + SHAP bg).
Evaluation	`churnguard/evaluate.py`	Metrics, figures, cost threshold, card, baseline.
Explain	`churnguard/explain.py`	Per-prediction SHAP mapped to business columns.
Predict	`churnguard/predict.py`	Inference + batch CSV CLI, shared by the API.
Monitor	`churnguard/monitor.py`	PSI data/prediction/performance drift.
Registry	`churnguard/registry.py`	File-based registry + champion/challenger.
Observability	`churnguard/observability.py`	JSON logs, Prometheus, rate limit, auth.
Serving	`churnguard/api.py`	FastAPI service + demo page.
Tracking	`churnguard/tracking.py`	Best-effort MLflow logging.

Two invariants that hold it together

- **No data leakage.** All scaling and one-hot encoding live *inside* a

scikit-learn `Pipeline`, fitted on the training fold only and cross-validated as one unit. Means, variances and categories never see the test data.

- **Train/serve parity.** The same `engineer_features()` runs at training time and

at request time, and inference reindexes incoming records to the exact `feature_names` the bundle was trained on — so the model never sees a differently-shaped input.

Part 4 — The Training Pipeline (select, tune, calibrate)

`train.py` is the heart of the project. It runs a disciplined, reproducible routine rather than fitting one model and hoping:

- **Select.** Three candidates — logistic regression, random forest, gradient

boosting — are each wrapped in `Pipeline(pre, clf)` and scored by 5-fold stratified cross-validated ROC-AUC. The highest mean score wins. On the shipped synthetic data logistic regression wins (CV AUC ~0.81).

- **Tune.** The winner gets a `RandomizedSearchCV` over a model-specific parameter

space (`n_iter=15`), still inside the leak-free pipeline and the same CV folds.

- **Calibrate.** The tuned estimator is wrapped in `CalibratedClassifierCV`

(isotonic) so predicted probabilities are honest. The *serving* model is the calibrated one; the *uncalibrated* `base_pipeline` is kept because SHAP needs the raw preprocessor + estimator, not the calibration wrapper.

- **Persist.** Everything is saved as one `ModelBundle`: the serving pipeline, the

base pipeline, the raw feature names, the transformed feature names, a 100-row transformed background sample for SHAP, provenance (version, trained-at, model name) and (after evaluation) the chosen threshold.

One fixed seed (`42`) flows through data generation, the train/test split, the CV shuffling and the search, so a run is reproducible; `requirements.lock` pins the exact dependency tree. MLflow logging is best-effort and never blocks training.

The cost-based threshold (chosen in evaluate.py)

`evaluate.py` does not leave the decision boundary at 0.5. `choose_threshold()` sweeps a 0.01–0.99 grid and, for each candidate, counts false negatives and false positives on the held-out test set, scoring `cost_fn * fn + cost_fp * fp` with the configured `5:1` ratio. The argmin is the threshold, and it is written *back into the bundle* so serving uses it. On the shipped data this lands at **0.20** and cuts expected cost from 1047 (at 0.5) to 654 — about 37% — catching 79% of real churners. Evaluation also writes `metrics.json`, six figures, a model card, the drift **reference profile** and the CI **baseline** — all from the same test split.

Part 5 — Operating the Model (monitoring, registry, observability)

ChurnGuard is built to be run in production, not just trained once.

Drift monitoring (monitor.py)

A deployed model decays as the world shifts under it. `monitor.py` compares a live batch to the reference profile captured at evaluation time using the **Population Stability Index (PSI)** — the standard in churn and credit scoring — computed per feature against the reference bucketing, plus prediction-score drift and (when labels are attached) performance drift measured as a ROC-AUC drop. PSI bands come from config (`< 0.10` stable, `0.10–0.25` warn, `> 0.25` alert). It is implemented in a few lines of NumPy with a self-contained JSON reference committed to the repo, so monitoring works the moment someone clones it — no server to stand up.

Model registry + champion/challenger (registry.py)

Every trained model is a versioned, immutable artifact with a lifecycle stage (`staging` -> `production` -> `archived`) recorded in a `registry.json` manifest. `evaluate_promotion()` is the champion/challenger decision: a challenger is promoted only if there is no incumbent, or it beats production on the configured metric by at least `min_improvement`. `scripts/retrain.py` is the scheduled job that trains, evaluates, registers a challenger and auto-promotes if it wins — promoting to production archives the old champion, so rollback is one call.

Observability + CI quality gate

`observability.py` adds standard-library-only production hardening: thread-safe request/latency/prediction counters with a Prometheus exposition and p50/p95/p99 latencies, one-line structured **JSON logs** with a request id, a fixed-window per-IP rate limiter, and optional `X-API-Key` auth (enabled only when `CHURNGUARD_API_KEY` is set, so demo use needs no secret). Finally, `scripts/check_regression.py` runs in CI and **fails the build** if a new model regresses beyond `registry.regression_tolerance` on ROC-AUC, PR-AUC or Brier vs. the committed baseline — making model quality a first-class CI check.

Part 6 — Tech Stack

Area	Choice
Language / runtime	Python 3.10–3.12
ML / data	scikit-learn, pandas, NumPy, SciPy, SHAP
Serving	FastAPI, Uvicorn, Pydantic v2
Explainability	SHAP (LinearExplainer / TreeExplainer)
Monitoring	PSI drift (data / prediction / performance)
MLOps	File-based registry, champion/challenger, CI regression gate
Observability	JSON logs, Prometheus text metrics, rate limit, API-key auth
Tracking	MLflow (best-effort, local `mlruns/`)
Packaging	Docker (trains at build), pip-tools `requirements.lock`
Quality	pytest + coverage (~91%), ruff, mypy, pre-commit
CI/CD	GitHub Actions (3.10/3.12 matrix + smoke + regression + scheduled retrain)

Part 7 — Honest Limitations

A mature engineer names the trade-offs:

- **The default dataset is synthetic.** The repo ships a generator that reproduces

the real Telco schema and churn drivers so it runs with zero downloads, but any *published* metric must come from `make data-real`. Synthetic numbers are never presented as real.

- **Single-snapshot training.** There is no automated concept-drift *alerting*

loop yet; `monitor` is run on demand or via `POST /monitor`, not wired to a scheduled batch.

- **In-memory observability.** Counters, latencies and the rate limiter live in

the process, so they reset on restart and do not aggregate across replicas — fine behind a Prometheus scrape and a gateway, not a substitute for them.

- **The cost ratio is an assumption.** `cost_fn:cost_fp = 5:1` is config-driven and

flagged as such; the chosen threshold is only as good as that ratio.

- **Promotion to production is human-gated by policy.** `retrain.py` can

auto-promote, but the project treats a live deploy as a stop-and-ask gate.

Part 8 — Interview Questions & Answers

A. Project overview

Q1. Give me the 30-second overview of ChurnGuard. ChurnGuard is an end-to-end churn-prediction service. It generates or loads data, engineers features in a leak-free scikit-learn pipeline, selects among three models by cross-validated ROC-AUC, tunes and calibrates the winner, and picks a decision threshold that minimises business cost rather than defaulting to 0.5. It serves calibrated probabilities with per-prediction SHAP explanations over a FastAPI API, and it ships the MLOps around that model: PSI drift monitoring, a file-based registry with champion/challenger promotion, structured observability and a CI regression gate.

Q2. Why churn, and why build the whole service instead of a notebook? For a subscription business, retention is far cheaper than acquisition, so ranking customers by risk lets a small budget be spent where it matters. The interesting part is not the classifier — it is making the output *trustworthy and operable*: calibrated probabilities, a cost-aware decision, explanations a retention team can act on, and the monitoring/registry/CI needed to run it past day one.

Q3. Is it making the retention decision for you? No. It is a scoring service, not an actuator. It emits a calibrated probability, a risk band, a cost-based decision and an explanation; a human or a downstream tool takes the action. The model card explicitly says it is not for automated consequential decisions without a human in the loop.

B. Architecture & design

Q4. Walk me through the pipeline end to end. `config.yaml` drives everything. `data.py` produces the frame; `features.py` adds derived columns and builds the `ColumnTransformer`; `train.py` cross-validates candidates, tunes and calibrates the winner and saves a `ModelBundle`; `evaluate.py` scores the held-out split, chooses the cost threshold, writes metrics, figures, the model card, the drift reference and the CI baseline; `predict.py` and `explain.py` do inference and SHAP; `api.py` serves it with observability wrapped around every request.

Q5. What is the ModelBundle and why does it exist? It is the single serialized artifact (`models/churnguard.joblib`). It carries the calibrated serving pipeline, the uncalibrated base pipeline (SHAP needs the raw preprocessor + estimator), the raw and transformed feature names, a background sample for SHAP, provenance, and the business threshold. Making the service one self-describing file means inference, explanation and the threshold all travel together — there is no second place to keep them in sync.

Q6. How is the whole thing config-driven, and why does that matter? Paths, candidate models, CV folds, the cost ratio, drift bands, the promotion metric and the rate limit all live in `config/config.yaml`; `config.py` is a typed, cached loader that resolves paths relative to the project root so the code runs from any working directory. No magic numbers in the code means a change is a config edit, reviewable in one place, and the same config governs CLI, tests, Docker and CI.

Q7. Why a file-based registry instead of a hosted MLflow Model Registry? No server to run, it is trivially testable, and it version-controls cleanly. A `registry.json` manifest beside immutable versioned bundles gives the provenance a team needs — what is in production and why — and promotion/rollback is a one-call manifest edit. The champion/challenger logic is the same a hosted registry would use; I just did not need the operational weight of a server for this scope.

C. ML / churn domain

Q8. How do you prevent data leakage? All preprocessing — median/most-frequent imputation, standard scaling, one-hot encoding — lives inside the scikit-learn `Pipeline`, and cross-validation runs the whole pipeline as one unit, so every fold fits its scalers and encoders on its own training rows only. Nothing learned from the test fold leaks into preprocessing. Identifier columns like `customerID` are dropped before modelling.

Q9. Why calibrate probabilities, and how? A raw classifier's "0.7" is often not a 70% chance, and a retention team needs the number to mean something to triage by it. I wrap the tuned model in `CalibratedClassifierCV` with isotonic regression and track the Brier score (0.13 on the shipped data) and a reliability curve to show the calibration holds. The calibrated model is what serves; the uncalibrated one is kept only for SHAP.

Q10. Explain the cost-based threshold. Why not 0.5? Missing a churner (false negative) costs more than a wasted retention offer (false positive) — I model it 5:1. `choose_threshold` sweeps the probability grid and picks the point minimising `cost_fn * fn + cost_fp * fp` on the test set. That lands at 0.20, which cuts expected cost from 1047 to 654 (~37%) and raises recall to 79%. 0.5 is only optimal when the two error types cost the same, which they rarely do.

Q11. How do the SHAP explanations stay business-readable? SHAP works in the model's transformed one-hot space. For a linear model the SHAP value of feature j is `coef_j * (x_j - E[x_j])`, computed in closed form; for a tree model it is `TreeExplainer`. I then map each transformed column back to its source column — summing the one-hot pieces of a categorical — so the explanation reads `Contract`, not `cat__Contract_Month-to-month`, and reports the top three drivers with a direction and the customer's actual value.

Q12. Why PR-AUC and recall-at-threshold, not just accuracy? Churn is imbalanced (~21% base rate), so accuracy is misleading — predicting "no churn" for everyone scores ~79%. PR-AUC (0.54 vs a 0.21 base) measures ranking on the positive class, and recall-at-the-chosen-threshold (0.79) is the business question: of the customers who will actually leave, how many did we flag in time to act.

D. Deep technical

Q13. How do you guarantee train/serve parity? One function, `engineer_features()`, runs in both paths and is deliberately tolerant of partial records, so a serving request with missing columns gets the same derivations. At inference `predict_records` reindexes the incoming frame to the bundle's exact `feature_names` before `predict_proba`, so column order and set always match what the model was fitted on; the pipeline's imputers fill anything absent.

Q14. How is PSI computed and graded? For a numeric feature I bucket the reference values into deduplicated quantile edges with open ends and store the reference percentages; a live batch is histogrammed into the same edges and PSI is `sum((actual - expected) * ln(actual/expected))` with an epsilon floor to avoid log(0). Categoricals compare category frequencies over the union of levels. Each feature, plus the prediction distribution and (with labels) the ROC-AUC drop, is graded ok/warn/alert from the config bands, and the report's overall status is the worst of them.

Q15. What happens on `POST /predict` if no model has been trained? `load_bundle` raises `ModelNotTrainedError`, which the endpoint translates to a 503 with a message telling you to train first — not a 500 stack trace. The bundle is loaded once and cached per path, with a `cache_clear` hook the tests and retraining use to force a reload after a new model is written.

Q16. Walk through the observability middleware. A FastAPI middleware assigns a 12-char request id, times the request, records it in the thread-safe `MetricsRegistry` (even on exception, as a 500), logs one JSON line with method/path/status/latency/client, and returns the id in an `X-Request-ID` header. The registry exposes `/stats` (throughput, p50/p95/p99) and `/metrics/prometheus`. `/predict` and `/monitor` additionally depend on a `guard` that enforces the per-IP rate limit and the optional API key.

Q17. How does the champion/challenger promotion decide? `evaluate_promotion` compares the candidate's metric (ROC-AUC by default) against the current `production` entry in the manifest. If there is no incumbent it promotes; otherwise the challenger must beat the champion by at least `min_improvement` to win. Promoting to production flips the old champion to `archived` in the same manifest write, so there is always exactly one production model and rollback is just re-promoting an older version.

E. Trade-offs & system design

Q18. Biggest trade-off you made? Shipping a synthetic dataset as the default. Pro: the repo trains, evaluates and serves with zero downloads, so CI and a fresh clone just work, and the generator reproduces the real schema and churn drivers from a logistic model so a classifier recovers a genuine signal. Con: the numbers are not real, so I gate any published metric behind `make data-real` and never present synthetic figures as real.

Q19. The observability is all in-memory. Why, and when does it break? It is standard-library only — no exporter or agent to deploy — which keeps the service light and the repo dependency-lean. It breaks across replicas and restarts: counters reset and do not aggregate horizontally. In real production it sits behind a Prometheus scrape (the exposition is there) and a gateway for durable auth and rate limiting; the in-process versions are a sensible default, not the final word.

Q20. How would you scale this to millions of customers? Serving is stateless, so it scales horizontally behind a load balancer; the batch cap is 1000/request by design. I would move scoring to a batched/async path, externalise metrics to a real Prometheus/push gateway, move rate limiting and auth to the gateway, and run monitoring as a scheduled job writing drift reports to storage with alerting wired to the warn/alert bands I already compute.

Q21. If the real cost ratio changes, what do you touch? Only `config.yaml`'s `cost_fn`/`cost_fp`, then re-run evaluate — `choose_threshold` recomputes the optimum and writes it back into the bundle, and the model card and cost curve regenerate. Nothing in the serving code changes, because the threshold lives in the artifact, not the code. Longer term I would online-retune it as the ratio drifts.

F. Behavioral

Q22. What are you most proud of technically? The discipline around *trust*: calibration plus a cost-based threshold plus business-readable SHAP, then the operations layer — PSI drift, a versioned registry with champion/challenger, and a CI gate that fails the build on a model-quality regression the same way a unit test does. It is the difference between a model that scores well in a notebook and one a team can actually run.

Q23. What was the hardest part? Keeping SHAP honest across model types and mapping it back to business columns. SHAP lives in transformed one-hot space, calibration wraps the estimator, and linear vs. tree models need different explainers. I had to keep the uncalibrated base pipeline in the bundle, branch the explainer on whether the estimator exposes `coef_`, and sum one-hot contributions back to their source column so the output speaks the business's language.

Q24. The project has a CLAUDE.md "constitution" and agent roles. What's that about? It documents how the project was built — a phase-by-phase protocol (tests first, parallel code and security review, an independent verifier, a human gate before each commit) with a sources-of-truth hierarchy and an anti-hallucination rule that logs SPEC-GAPs instead of guessing. It is the engineering process made explicit, and the shipped architecture mirrors that phase map.

Q25. What would you do next? Wire concept-drift *alerting* to a scheduled monitoring batch, online-retune the threshold as the real cost ratio moves, A/B test the retention actions the scores trigger to measure realised ROI, and add DVC data versioning with a small dashboard over the batch scores.

Q26. What did you learn? That for an ML service the model is the easy part; the durable value is calibration, a cost-aware decision, explanations, and the MLOps scaffolding — leakage control, reproducibility, drift monitoring, a registry and a CI quality gate — that let a team trust and operate it over time.

Quick Facts (cheat sheet)

- **What:** End-to-end churn-prediction service — ML pipeline + FastAPI.
- **Flow:** data -> features -> train -> evaluate -> serve -> monitor -> registry.
- **Model:** 3 candidates by CV ROC-AUC -> tune (randomized) -> calibrate (isotonic).
- **Decision:** cost-based threshold (cost_fn:cost_fp = 5:1), chosen at ~0.20, not 0.5.
- **Results (synthetic):** ROC-AUC 0.81, PR-AUC 0.54, Brier 0.13, recall 0.79.
- **Explainability:** per-prediction SHAP mapped back to business columns.
- **Ops:** PSI drift monitor, file-based registry + champion/challenger, CI regression gate.
- **Hardening:** JSON logs, Prometheus metrics, per-IP rate limit, optional API-key auth.
- **Stack:** Python 3.10–3.12, scikit-learn, FastAPI, SHAP, Docker; 43 tests, ~91% coverage.
- **License:** MIT · one source of truth in `config/config.yaml`.